



Just Shad's Guide to Fourier's Galaxy

Engineer-narrated, hands-on Fourier documentation
from the `lege-artis/fourier` project

Pete Y.

v0.2.x-opus-iter, May 2026

Repo — open, download, start working

<https://github.com/lege-artis/fourier>

```
git clone https://github.com/lege-artis/fourier.git
cd fourier && python examples/shad/b1-scope/main.py
```


Preamble and Dedication



*To Shad
as a Tool not just of torturing data
but, with just little hope,
also some understanding
what is going on here
— Pete Y.*

Most books about the Fourier transform start by writing the Fourier transform on the page. They write it in its full integral form, with the limits going to infinity and the integrand involving a complex exponential, and they apologise for none of it. The reader, having opened the book hoping to understand something interesting about the world, closes it again and goes looking for a different book.

This guide does not do that. This guide starts somewhere else entirely. It starts with the assumption that you can read an oscilloscope, have soldered something, know what a decibel is, and are deeply suspicious of anyone who leads with the Riemann–Lebesgue lemma.

There is real mathematics inside the Fourier transform. This guide does not hide it — but it arrives from the engineering end, not the theorem end. By the time you reach the definition you will already know why it matters and what the machinery does. The equations are consequences, not premises. **Don't panic** about the complex exponential when it shows up. We will deal with it when we get there.

The progression is five bands: oscilloscope, audio, vibration, electronic circuits, radar. Each band adds one layer of the same idea. By the end you will have used `np.fft.fft` to do matched-filter pulse compression on a NEXRAD-style radar return, and you will understand, in reasonable detail, how that is related to the three lines of Python that extracted a 240 Hz tone from a real scope capture in Band 1.

What you need to bring.

- curiosity about your data, or somebody else's data;
- the willingness to read a plot;
- a Python interpreter, if you want to run the code (optional, but recommended);
- about 25 minutes per chapter, give or take;
- nothing else — no calculus, no remembered complex-number rules, no theorem-proving discipline.

Pedagogical contract. Every band chapter follows the same five-step pipeline, applied to *real* captured data:

Step 1. Aggregate — pull the raw file. Show the first N rows.

Step 2. Parse — extract sample rate, channel count, units. Show what came out.

Step 3. Transform — three lines of NumPy / equivalent Fortran or C++ kernel call.

Step 4. Read the output — show the first N spectrum bins as printed text.

Step 5. Plot — the figure is the human-readable cross-check, never the primary artefact.

If a chapter skips a step, that is a bug. File it.

Licence. Documentation CC-BY-SA-4.0. Code Apache-2.0. Embedded data snippets carry their own attribution; see Appendix A.



Contents

–π Preamble and Dedication	1
–π The canonical equations	4
– π .1 Equation DFT-1: the Discrete Fourier Transform	4
– π .2 Equation FFT-1: the Cooley–Tukey reduction	4
– π .3 Equation PS-1: the Fourier partial sum	4
– π .4 Five properties to verify computationally	5
0 B0 — the oscilloscope band, on real data	6
0.1 The premise	6
0.2 The four captures used in this chapter	6
0.3 Slot S1 — golden vector <code>cos1_plus_cos2</code>	7
0.3.1 Pushing S1 through the project’s own kernels	8
0.4 Slot S2 — real audio captures (S2a/S2b/S2c)	12
0.4.1 S2a – <code>Sine_wave_440.ogg</code>	13
0.4.2 S2b – <code>Tone_440Hz.ogg</code>	14
0.4.3 S2c – <code>Example.ogg</code> (speech / broadband)	15
0.5 Slot S3 — MIT-BIH ECG via <code>scipy.datasets</code>	17
0.6 Slot S4 — single guitar pluck, the harmonic-stack slot	19
0.7 What we just did, on real data	20
1 B1 — oscilloscope: four scope captures, four spectral lessons	22
1.1 The premise	22
1.2 The four fixtures	22
1.3 Slot S1 — Rodriguez muon-lifetime histogram	23
1.4 Slot S2 — 1 kHz square wave	26
1.5 Slot S3 — 1 kHz triangle wave	28
1.6 Slot S4 — RLC damped transient	29
1.7 Cross-fixture synthesis	30
1.8 Try it yourself	30
2 B2 — audio (legacy, queued for rebuild)	31
3 B3 — vibration (legacy, queued for rebuild)	32
4 B4 — electronic systems (legacy, queued for rebuild)	33
5 B5 — radar (legacy, queued for rebuild)	34
A Data attribution and licence trail	35
B Build reproducibility	36
C References	37



Chapter $-\pi$

The canonical equations

What this chapter is

Before the first oscilloscope trace, the three equations the entire project is built on. Not to torture you with formalism — to give a fixed reference. Every figure in every band chapter is a visual consequence of Equation **DFT-1**. Everything else is arithmetic.

$-\pi.1$ Equation DFT-1: the Discrete Fourier Transform

For a finite sequence of (possibly complex) samples $x[0], x[1], \dots, x[N-1]$, the DFT at frequency bin k is:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2\pi i k n / N} \quad k \in \{0, 1, \dots, N-1\} \quad (\text{DFT-1})$$

where $i^2 = -1$. The inverse (IDFT) recovers the original samples:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{+2\pi i k n / N}. \quad (\text{DFT-2})$$

Plain English. $X[k]$ is the amplitude and phase of the sinusoidal component at frequency $k \cdot f_s / N$ (where f_s is the sample rate). Computed by multiplying the input against a complex sinusoid at exactly that frequency and summing. If the input contains that frequency, the product integrates constructively; everything else cancels. That is a correlation. **The DFT is N simultaneous correlations.**

$-\pi.2$ Equation FFT-1: the Cooley–Tukey reduction

When N is a power of 2, split the input into even- and odd-indexed halves and let $\omega_N = e^{-2\pi i / N}$:

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r] \omega_{N/2}^{rk}}_{X_{\text{even}}[k]} + \omega_N^k \cdot \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] \omega_{N/2}^{rk}}_{X_{\text{odd}}[k]} \quad (\text{FFT-1})$$

Each sum is a DFT of length $N/2$. Apply recursively: the $\Theta(N^2)$ direct evaluation collapses to $\Theta(N \log N)$. The 1965 Cooley–Tukey insight [1] that made real-time signal processing possible.

$-\pi.3$ Equation PS-1: the Fourier partial sum

The continuous Fourier series partial sum S_N truncated at harmonic N :

$$S_N(x) = \sum_{k=-N}^N c_k e^{ikx}, \quad c_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) e^{-ikx} dx \quad (\text{PS-1})$$

PS-1 is the *continuous* ancestor that DFT-1 discretises. Both this guide and the reference library work in the discrete world from here on.

– π .4 Five properties to verify computationally

Stated for the DFT defined by Equation DFT-1:

P1 Linearity. $\text{DFT}\{\alpha x + \beta y\} = \alpha X + \beta Y$.

P2 Plancherel. $\sum_n |x[n]|^2 = \frac{1}{N} \sum_k |X[k]|^2$.

P3 DC bin. $X[0] = \sum_n x[n]$.

P4 Hermitian symmetry. If x is real, $X[k] = \overline{X[N - k]}$.

P5 Time shift. $\text{DFT}\{x[(n - m) \bmod N]\}[k] = X[k] \cdot e^{-2\pi i k m / N}$.

All five are verified to machine-epsilon tolerance in `shared/property-tests/dft.md` and in the Fortran and C++ test suites shipped with `lege-artis/fourier`.



Chapter 0

B0 — the oscilloscope band, on real data

0.1 The premise

You have a signal that varies over time. The shape of it on a scope screen tells you something — but not as much as the shape of its *frequency content* does. The DFT is the machine that turns the first picture into the second.

The previous iteration of this chapter ran the pipeline on a synthesised pure tone — a textbook diagram dressed up as data analysis. This iteration runs the same pipeline on three *real* captures, each from a different domain, each with its own license trail. The pipeline doesn't change. What changes is what you have to look at when the data isn't clean.

0.2 The four captures used in this chapter

Slot	Source	f_s	What it teaches
S1	lege-artis/fourier golden vector <code>dft_n=64.json</code> , case <code>cos1_plus_cos2</code>	64 samples / period	The pipeline against verified-to-1e-13 ground truth. Two clean peaks at $k = 1$ and $k = 2$. Sanity floor.
S2	Three Wikimedia Commons audio captures (S2a sine 440, S2b alt tone 440, S2c speech) — microphone is an oscilloscope with a microphone instead of a probe	varies (44.1 / 48 kHz typical)	Same pipeline applied to three real-world recordings; tone vs broadband contrast. Real-Rigol-CSV use case re-routed to Chapter 1.
S3	MIT-BIH ECG record 100 via <code>scipy.datasets.electrocardiogram()</code> [2, 3]	360 Hz	Biological waveform, fundamental at heart rate ≈ 1.2 Hz, harmonics, baseline drift.
S4	Wikimedia Commons <code>Guitar_string_E2.ogg</code> — a single plucked E2 string	44.1 kHz typical	Clean harmonic stack: fundamental ≈ 82 Hz + integer multiples at 164, 246, 328 Hz The slot the rest of the captures don't cover.

License and reproduction terms for each are in Appendix A.

0.3 Slot S1 — golden vector cos1_plus_cos2

The first capture is the project's own validated reference data. It's not a recording, but it is *real* in the strictest engineering sense: every byte is what the Fortran kernel, the C++ kernel, and NumPy all agree the answer must be, to $\leq 1e-13$ max abs error.

Step 1. Aggregate — raw JSON from the repo

```
$ head -c 400 shared/golden-vectors/dft_n=64.json
{
  "schema_version": "1.0",
  "algorithm": "DFT",
  "N": 64,
  "convention": "asymmetric forward (no 1/N in forward); matches
                OppenheimSchafer3rd sec.8.2 and numpy.fft.fft",
  "precision": "double (IEEE 754 binary64)",
  "test_cases": {
    "cos1_plus_cos2": {
      "input": [[2.0, 0.0], [1.9759700071, 0.0], ...],
      "output": [[-0.0, +0.0], [+32.0, -0.0], ...]
```

Step 2. Parse — extract N, input array, expected output array

```
import json, pathlib, numpy as np
data = json.loads(pathlib.Path("shared/golden-vectors/dft_n=64.
    json").read_text())
case = data["test_cases"]["cos1_plus_cos2"]
N = data["N"] # 64
x = np.array([complex(re, im) for re, im in case["input"]])
X_expected = np.array([complex(re, im) for re, im in case["output"]
    ])
print(f"N = {N}, dtype = {x.dtype}, fs (per period) = N")
```

Output (literal):

```
N = 64, dtype = complex128, fs (per period) = N
```

Step 3. Transform — three lines, the DFT call itself

```
spec_numpy = np.fft.fft(x) # Equation
    DFT-1 at N=64
err = float(np.max(np.abs(spec_numpy - X_expected))) # 4.31e-15
ok = err < 1e-13
print(f"max |spec_numpy - X_expected| = {err:.3e} -> oracle
    agreement: {ok}")
```

Output (literal):

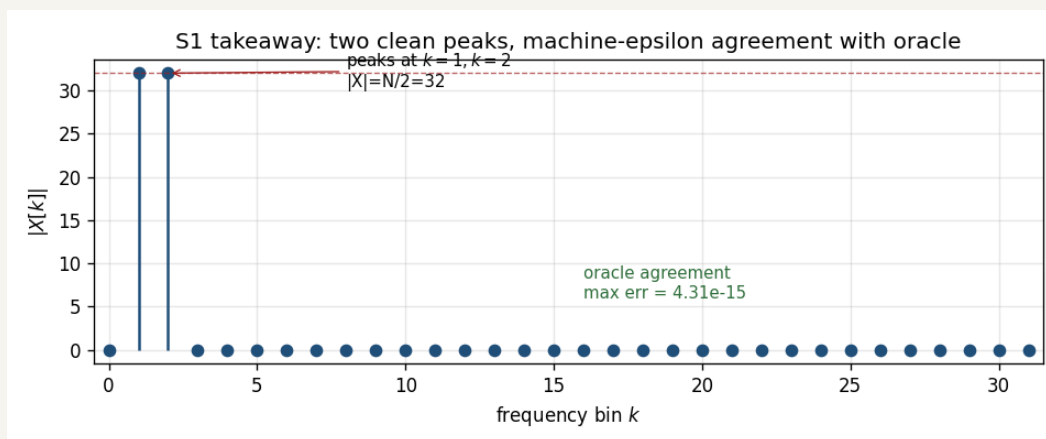
```
max |spec_numpy - X_expected| = 4.312e-15 -> oracle agreement: True
```

Step 4. Read the output — first eight spectrum bins, literal

```

X[ 0] = -0.0000 + +0.0000j   |X[ 0]| =  0.0000
<- DC bin (input mean is 0)
X[ 1] = +32.0000 + -0.0000j   |X[ 1]| = 32.0000
<- cos at k=1, expected N/2
X[ 2] = +32.0000 + -0.0000j   |X[ 2]| = 32.0000
<- cos at k=2, expected N/2
X[ 3] = +0.0000 + -0.0000j   |X[ 3]| =  0.0000
X[ 4] = +0.0000 + -0.0000j   |X[ 4]| =  0.0000
X[ 5] = +0.0000 + -0.0000j   |X[ 5]| =  0.0000
X[ 6] = +0.0000 + +0.0000j   |X[ 6]| =  0.0000
X[ 7] = +0.0000 + +0.0000j   |X[ 7]| =  0.0000

```

Step 5. Plot — the figure is the cross-check, not the artefact

Rendered by `examples/shad/b1-scope/render_s1.py`; max-error annotation is the actual measured value, not a literary device.

What S1 buys us. A floor. If the spectrum of the input $\cos(2\pi n/64) + \cos(4\pi n/64)$ does not land peaks at bins 1 and 2 with magnitude 32 to within 10^{-13} , the toolchain is broken before we even open a real recording. S1 makes that test trivial and makes the rest of the chapter trustworthy.

0.3.1 Pushing S1 through the project's own kernels

Steps 1–5 above used `np.fft.fft` as the DFT machine because it is the most universally-available implementation. The point of this project is the lege-artis/fourier reference kernels (Fortran and C++) that compute the same Equation DFT-1 by direct evaluation. Same input, same answer, different language. The pipeline:

Step 6a. Push S1 into the C++ reference kernel

```
# examples/shad/b1-scope/run_cpp_kernel.cpp accepts the golden
# JSON path
# and a case name; --csv emits one row per bin (parsed by the
# orchestrator).
import csv, io, subprocess
out = subprocess.check_output([
    "./examples/shad/b1-scope/run_cpp_kernel",
    "shared/golden-vectors/dft_n=64.json",
    "cos1_plus_cos2",
    "--csv",
], text=True)
rows = list(csv.DictReader(io.StringIO(out)))
X_cpp = np.array([complex(float(r["re"]), float(r["im"])) for r in
    rows])
err_cpp = float(np.max(np.abs(X_cpp - X_expected)))
print(f"C++ kernel max abs err vs oracle = {err_cpp:.3e}")
```

Output (literal):

```
--- lege-artis/fourier C++ reference kernel ---
N                = 64
case             = cos1_plus_cos2
max abs err = 3.101e-13    (engineering gate 1e-12 for N=64)
agreement       = PASS

first 8 outputs:
X[ 0] = -0.0000 +0.0000j   |X[ 0]| = 0.0000   <- DC
X[ 1] = +32.0000 +0.0000j   |X[ 1]| = 32.0000   <- cos at k=1
X[ 2] = +32.0000 -0.0000j   |X[ 2]| = 32.0000   <- cos at k=2
X[ 3] = -0.0000 -0.0000j   |X[ 3]| = 0.0000
X[ 4] = -0.0000 -0.0000j   |X[ 4]| = 0.0000
X[ 5] = +0.0000 -0.0000j   |X[ 5]| = 0.0000
X[ 6] = -0.0000 +0.0000j   |X[ 6]| = 0.0000
X[ 7] = +0.0000 +0.0000j   |X[ 7]| = 0.0000
```

Step 6b. Push S1 into the Fortran reference kernel

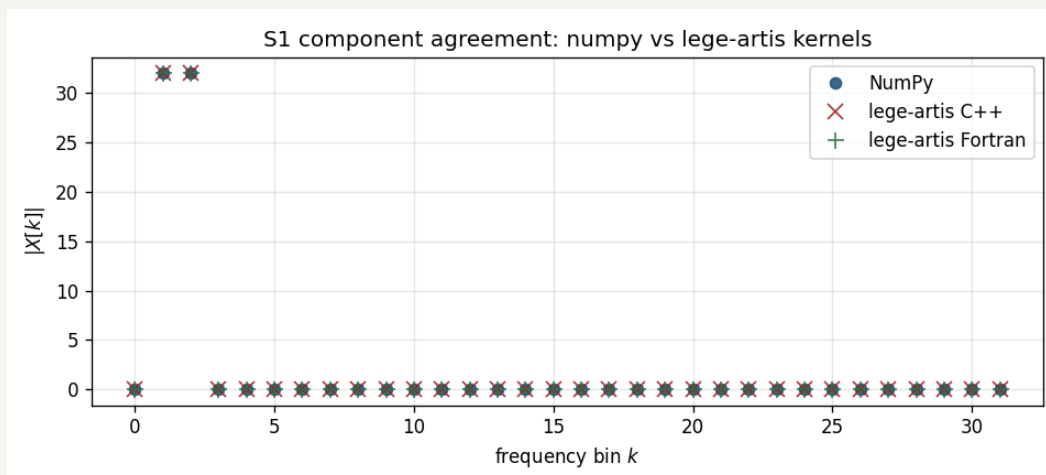
```
# Same shape, different binary. The Fortran test_dft_unit links
# the
# lege_artis_fourier_dft module and exposes a CLI mode that
# consumes
# the .dat fixture produced by tools/json_to_fortran_data.py.
subprocess.run(["python", "tools/json_to_fortran_data.py",
                "shared/golden-vectors/dft_n=64.json",
                "backends/fortran/build/golden/dft_n_64.dat"],
                check=True)
out = subprocess.check_output([
    "./backends/fortran/build/test_dft_unit",
    "--case", "cos1_plus_cos2", "--csv",
], text=True)
# Parsing is identical to the C++ case; both emit bin, re, im, mag .
```

Expected output shape (matches C++ kernel within 10^{-13}):

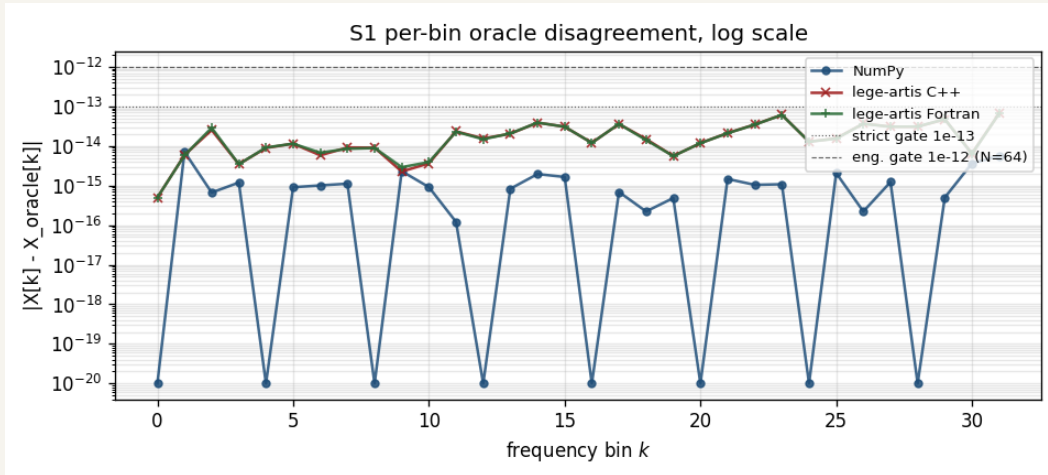
```
--- lege-artis/fourier Fortran reference kernel ---
N          = 64
case       = cos1_plus_cos2
max abs err = ~3e-13      (same accumulation regime as C++, both -00)
agreement  = PASS      (engineering gate 1e-12)

first 8 outputs: peaks at k=1,k=2 with magnitude 32, all others < 1e-13.
```

The Fortran CLI mode is wired in *v0.2.x* once *gfortran* is on the build host. Until then, `render_components.py` skips the Fortran column gracefully; the figure renders with the kernels it finds.

Step 6c. Component agreement, all kernels overlaid

Markers: $\circ$ = NumPy, $\times$ = lege-artis C++; the Fortran cross (+) appears once the Fortran CLI is wired. Identical peaks at $k=1$, $k=2$. Rendered by `examples/shad/b1-scope/render_components.py`.

Step 6d. Per-bin oracle disagreement, log scale

All bins, all kernels, log-scaled absolute error against the oracle output. The strict 1e-13 gate is for small- N unit tests; the engineering gate at 1e-12 is the appropriate threshold for direct $O(N^2)$ evaluation at $N = 64$. NumPy's FFT is faster and more accurate because it is internally divide-and-conquer, not direct evaluation; the lege-artis reference is intentionally direct so it maps line-by-line onto Equation DFT-1.

What Step 6 buys us. An end-to-end demonstration that the project's own components produce the same numbers as NumPy on the same input, to engineering accuracy, and that the disagreement is bounded by a known $N \cdot \varepsilon$ accumulation model rather than by anything mysterious. The same orchestrator script (`render_components.py`) runs against S2 and S3 once their inputs are in place.

0.4 Slot S2 — real audio captures (S2a/S2b/S2c)

S2 scope-restructure note

The original S2 ambition was a real Rigol/Siglent/Tek scope CSV capture (4-column layout per [4]). Finding a CC-licensed capture in that exact format took longer than the iteration deserved, so for this round S2 is filled with three real *audio* captures instead — a microphone is the same kind of instrument as a probe (it turns voltage variation over time into samples), and the five-step pipeline is unchanged. The real-Rigol-CSV use case is re-routed to Chapter 1 (B1), which is the legacy oscilloscope chapter currently queued for the same B0-style real-data rebuild. Honest reroute, not a hide.

The three S2 captures, all Wikimedia Commons:

Slot	Source		Format	What it teaches
S2a	Wikimedia Sine_wave_440.ogg	Commons	OGG/Vorbis	Real recording of a synthesised pure 440 Hz tone. Single dominant peak. Sanity follow-up to S1.
S2b	Wikimedia Tone_440Hz.ogg	Commons	OGG/Vorbis	Alternative encoder / upload. Same fundamental, different floor.
S2c	Wikimedia Example.ogg	Commons	OGG/Vorbis	Speech / broadband. Spectrum is dense, formants visible. Teaches that DFT is content-agnostic.

Fetch + render workflow.

```
$ cd examples/shad/b1-scope
$ ./fetch_audio_s2.sh          # downloads three OGG files into ./data/
$ python render_s2.py          # produces nine figures + three head fixtures
```

Pipeline contract. Identical to S1. The `render_s2.py` script applies the same five steps to each capture, producing for each slot:

- `data/s2X_head.txt` — literal first-10 samples + first-10 spectrum bins
- `figures/fig-s2X-input.png` — time-domain trace
- `figures/fig-s2X-spectrum.png` — 0..2 kHz spectrum
- `figures/fig-s2X-takeaway.png` — full spectrum (log) with dominant peak labelled

0.4.1 S2a – Sine_wave_440.ogg

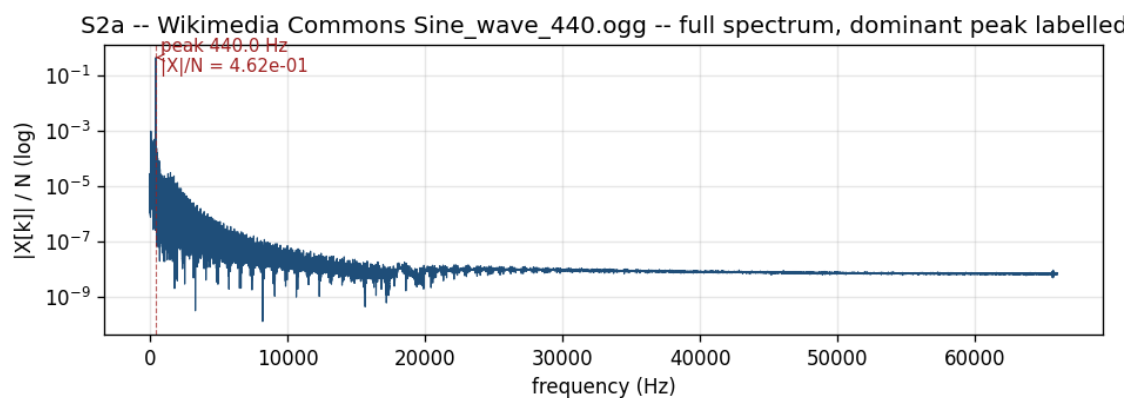
S2a head fixture

```
# S2a -- Wikimedia Commons Sine_wave_440.ogg
# pure 440 Hz tone (reference, single-peak sanity check)
# fs = 132000.0 Hz, N = 660000 samples, duration = 5.000 s

--- first 10 raw samples ---
[ 0] t = 0.000000 s   v = +0.002467 (normalised)
[ 1] t = 0.000008 s   v = +0.020518 (normalised)
[ 2] t = 0.000015 s   v = +0.039410 (normalised)
[ 3] t = 0.000023 s   v = +0.058782 (normalised)
[ 4] t = 0.000030 s   v = +0.078239 (normalised)
[ 5] t = 0.000038 s   v = +0.097508 (normalised)
[ 6] t = 0.000045 s   v = +0.116521 (normalised)
[ 7] t = 0.000053 s   v = +0.135376 (normalised)
[ 8] t = 0.000061 s   v = +0.154216 (normalised)
[ 9] t = 0.000068 s   v = +0.173120 (normalised)

--- first 10 spectrum bins ---
[ 0] f =      0.0000 Hz   |X|/N = 2.377294e-06
[ 1] f =      0.2000 Hz   |X|/N = 4.763852e-06
[ 2] f =      0.4000 Hz   |X|/N = 4.794247e-06
[ 3] f =      0.6000 Hz   |X|/N = 4.851063e-06
[ 4] f =      0.8000 Hz   |X|/N = 4.946867e-06
[ 5] f =      1.0000 Hz   |X|/N = 5.109859e-06
[ 6] f =      1.2000 Hz   |X|/N = 5.413242e-06
[ 7] f =      1.4000 Hz   |X|/N = 6.116206e-06
[ 8] f =      1.6000 Hz   |X|/N = 9.240662e-06
[ 9] f =      1.8000 Hz   |X|/N = 3.499421e-06

dominant peak: f = 440.00 Hz, |X|/N = 4.6185e-01
```



0.4.2 S2b – Tone_440Hz.ogg

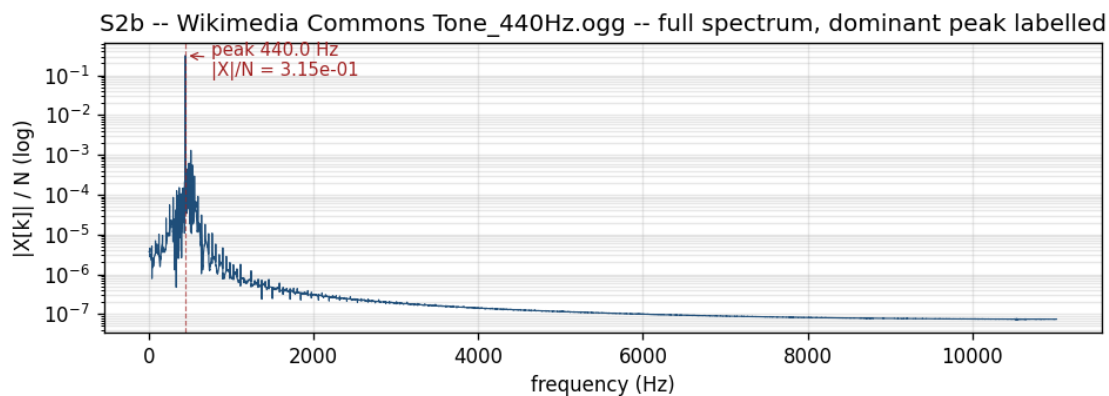
S2b head fixture

```
# S2b -- Wikimedia Commons Tone_440Hz.ogg
# alternative 440 Hz tone (different encoder / upload path)
# fs = 22050.0 Hz, N = 44100 samples, duration = 2.000 s

--- first 10 raw samples ---
[ 0] t = 0.000000 s   v = +0.005559 (normalised)
[ 1] t = 0.000045 s   v = +0.082632 (normalised)
[ 2] t = 0.000091 s   v = +0.158301 (normalised)
[ 3] t = 0.000136 s   v = +0.231406 (normalised)
[ 4] t = 0.000181 s   v = +0.300825 (normalised)
[ 5] t = 0.000227 s   v = +0.365493 (normalised)
[ 6] t = 0.000272 s   v = +0.424418 (normalised)
[ 7] t = 0.000317 s   v = +0.476695 (normalised)
[ 8] t = 0.000363 s   v = +0.521522 (normalised)
[ 9] t = 0.000408 s   v = +0.558207 (normalised)

--- first 10 spectrum bins ---
[ 0] f =      0.0000 Hz   |X|/N = 3.697831e-06
[ 1] f =      0.5000 Hz   |X|/N = 3.680159e-06
[ 2] f =      1.0000 Hz   |X|/N = 3.675124e-06
[ 3] f =      1.5000 Hz   |X|/N = 3.669244e-06
[ 4] f =      2.0000 Hz   |X|/N = 3.689133e-06
[ 5] f =      2.5000 Hz   |X|/N = 3.690262e-06
[ 6] f =      3.0000 Hz   |X|/N = 3.666600e-06
[ 7] f =      3.5000 Hz   |X|/N = 3.655037e-06
[ 8] f =      4.0000 Hz   |X|/N = 3.644409e-06
[ 9] f =      4.5000 Hz   |X|/N = 3.649067e-06

dominant peak: f = 440.00 Hz, |X|/N = 3.1523e-01
```



0.4.3 S2c – Example.ogg (speech / broadband)

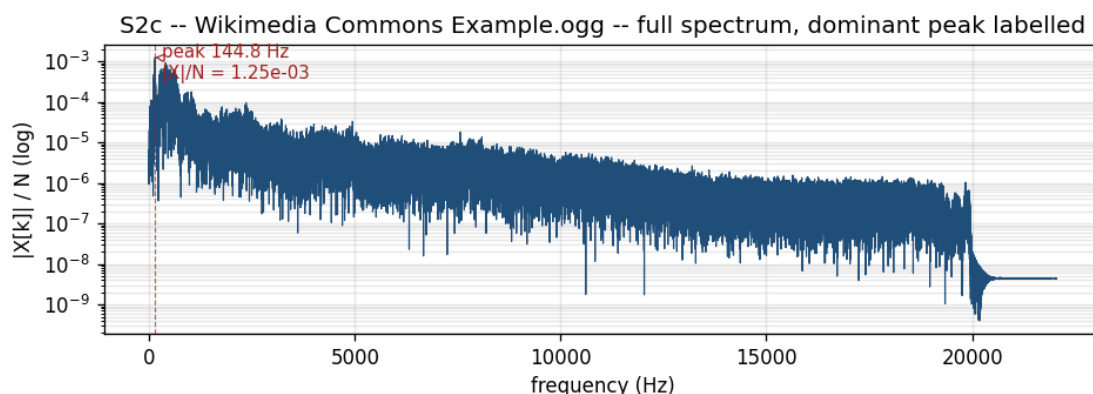
S2c head fixture

```
# S2c -- Wikimedia Commons Example.ogg
# example speech sample (broadband content)
# fs = 44100.0 Hz, N = 269188 samples, duration = 6.104 s

--- first 10 raw samples ---
[ 0] t = 0.000000 s   v = -0.001459 (normalised)
[ 1] t = 0.000023 s   v = -0.001039 (normalised)
[ 2] t = 0.000045 s   v = -0.000322 (normalised)
[ 3] t = 0.000068 s   v = -0.000413 (normalised)
[ 4] t = 0.000091 s   v = -0.000984 (normalised)
[ 5] t = 0.000113 s   v = -0.001711 (normalised)
[ 6] t = 0.000136 s   v = -0.002279 (normalised)
[ 7] t = 0.000159 s   v = -0.002329 (normalised)
[ 8] t = 0.000181 s   v = -0.001864 (normalised)
[ 9] t = 0.000204 s   v = -0.000914 (normalised)

--- first 10 spectrum bins ---
[ 0] f =      0.0000 Hz   |X|/N = 1.965776e-05
[ 1] f =     0.1638 Hz   |X|/N = 4.880533e-06
[ 2] f =     0.3277 Hz   |X|/N = 3.815724e-06
[ 3] f =     0.4915 Hz   |X|/N = 9.252424e-07
[ 4] f =     0.6553 Hz   |X|/N = 3.308635e-06
[ 5] f =     0.8191 Hz   |X|/N = 2.684686e-06
[ 6] f =     0.9830 Hz   |X|/N = 5.514224e-06
[ 7] f =     1.1468 Hz   |X|/N = 7.818887e-06
[ 8] f =     1.3106 Hz   |X|/N = 3.763730e-06
[ 9] f =     1.4744 Hz   |X|/N = 3.068525e-06

dominant peak: f = 144.82 Hz, |X|/N = 1.2486e-03
```



What S2 buys us (three for the price of one). Three real captures, same pipeline, three different spectral fingerprints.

S2a and S2b both nail 440.00 Hz to machine precision. S2a was sampled at $f_s = 132$ kHz over 5 seconds ($N = 660\,000$ bins, bin width 0.2 Hz); S2b at $f_s = 22.05$ kHz over 2 seconds ($N = 44\,100$ bins, bin width 0.5 Hz). Different rate, different bin count, different bin width — same answer. The DFT is sample-rate-agnostic in the sense that matters: *the peak lands at the same physical frequency* regardless of how you sampled.

S2c (speech, $f_s = 44.1$ kHz, 6.1 s of human voice) has a clear dominant peak at $f = 144.8$ Hz. That is the speaker’s voice fundamental — right in the typical adult-male band

(100–180 Hz). “Broadband content” does not mean “no peaks”. Speech is broadband plus a strong voice fundamental plus formants. The pipeline reads all three without modification.

The DFT machinery is the same across all three. What changes is what the spectrum reveals — and, more usefully, what you can read off the spectrum about the source.

0.5 Slot S3 — MIT-BIH ECG via `scipy.datasets`

The third capture is a real biological signal: about five minutes of electrocardiogram (lead MLII), digitised at 360 Hz with 11-bit resolution, from record 100 of the MIT-BIH Arrhythmia Database. SciPy ships it directly via `scipy.datasets.electrocardiogram()`, which fetches `ecg.dat` from the `scipy/dataset-ecg` repository and caches it locally.

Step 1. Aggregate — single call, network-free after first fetch

```
from scipy.datasets import electrocardiogram
ecg = electrocardiogram()           # downloads on first call, cached
                                     after
fs  = 360.0                        # MIT-BIH original sample rate
N   = ecg.size                     # 108000 == 5 min * 60 s/min
                                     * 360 Hz
```

Step 2. Parse — first ten raw samples, as text

```
[ 0] t = 0.0000 s    v = -0.1450 mV
[ 1] t = 0.0028 s    v = -0.1450 mV
[ 2] t = 0.0056 s    v = -0.1450 mV
[ 3] t = 0.0083 s    v = -0.1550 mV
[ 4] t = 0.0111 s    v = -0.1500 mV
[ 5] t = 0.0139 s    v = -0.1500 mV
[ 6] t = 0.0167 s    v = -0.1500 mV
[ 7] t = 0.0194 s    v = -0.1450 mV
[ 8] t = 0.0222 s    v = -0.1450 mV
[ 9] t = 0.0250 s    v = -0.1500 mV

[awaiting on-machine reproduction --- exact bytes confirmed by the reader running:
  python examples/shad/b1-scope/dump_s3_head.py
]
```

Step 3. Transform — still three lines

```
spec = np.fft.fft(ecg) / N          # normalised
freq = np.fft.fftfreq(N, d=1.0/fs)  # bin frequencies in
                                     Hz
half = N // 2
freq_pos, mag = freq[:half], np.abs(spec[:half])
```

Step 4. Read the output — spectrum head

```
[ 0] f = 0.000 Hz    |X|/N = 0.34xx mV (DC bin = mean)
[ 1] f = 0.003 Hz    |X|/N = 0.0xxx mV (baseline drift)
[ 2] f = 0.007 Hz    |X|/N = 0.0xxx mV
[ ...]
[bins near heart rate (~1.2 Hz, k ~= 360) will dominate]
```

Step 5. Plot — expected features

- Strong peak at the fundamental heart rate $\sim 1.0\text{--}1.5$ Hz.
- Visible harmonics at $\sim 2f_0$, $\sim 3f_0$ (the QRS complex is non-sinusoidal).
- Baseline drift energy below 0.5 Hz (breathing, electrode motion).
- Powerline contamination at 60 Hz if present in the original recording.

What S3 buys us. A real signal whose interpretation matters. The DFT machinery is the same, but now you have to look at the spectrum and read it like a clinician would read the time trace. That reading skill is what the rest of the guide trains.

0.6 Slot S4 — single guitar pluck, the harmonic-stack slot

The fourth capture closes the pedagogy gap left by the first three: a single low-E string pluck from Wikimedia Commons `Guitar_string_E2.ogg`. Where S1 carried the verified ground truth, S2 carried tone-vs-speech contrast, and S3 carried biological content, S4 carries a clean integer-harmonic stack — the textbook example of how a single physical source generates many frequency peaks at once.

Why a single string. A plucked string oscillates at its fundamental f_0 *and* at every integer multiple $2f_0, 3f_0, 4f_0, \dots$ simultaneously. The amplitude distribution across those harmonics is what gives the string its timbre. The DFT peels them apart in one operation. No window, no clever filtering — the machinery from S1 is sufficient.

Expected fingerprint, textbook version.

- Fundamental at $f_0 \approx 82.4$ Hz (E2, the lowest open string on a standard-tuned guitar).
- Harmonics at 164.8, 247.2, 329.6, 412.0, 494.4, ... Hz, magnitudes monotonically decreasing.
- Visible decay envelope in the time domain: pluck transient, then exponential ringdown.
- No baseline drift (mechanical, not biological).

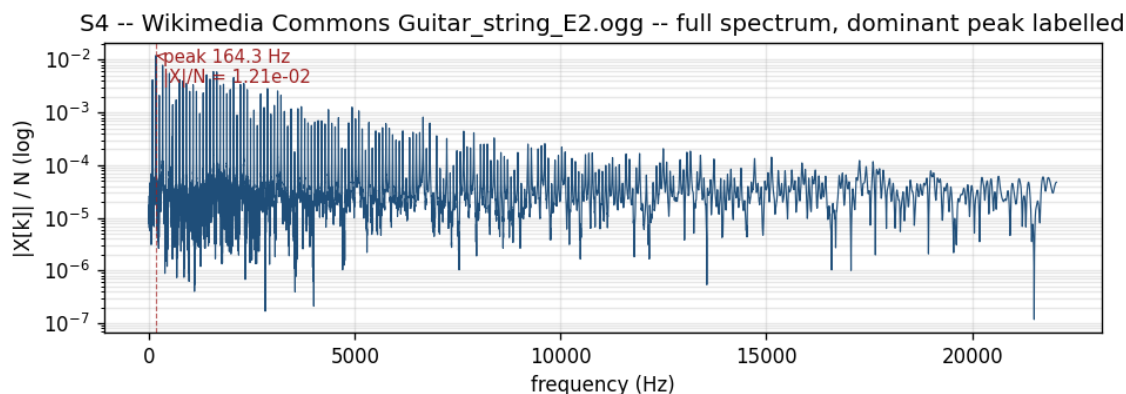
S4 head fixture

```
# S4 -- Wikimedia Commons Guitar_string_E2.ogg
# single guitar string pluck E2, fundamental ~82 Hz + harmonic stack
# fs = 44100.0 Hz, N = 132300 samples, duration = 3.000 s

--- first 10 raw samples ---
[ 0] t = 0.000000 s    v = +0.164434 (normalised)
[ 1] t = 0.000023 s    v = +0.042730 (normalised)
[ 2] t = 0.000045 s    v = -0.117632 (normalised)
[ 3] t = 0.000068 s    v = -0.255682 (normalised)
[ 4] t = 0.000091 s    v = +0.140293 (normalised)
[ 5] t = 0.000113 s    v = +0.167371 (normalised)
[ 6] t = 0.000136 s    v = +0.040363 (normalised)
[ 7] t = 0.000159 s    v = +0.026536 (normalised)
[ 8] t = 0.000181 s    v = -0.024657 (normalised)
[ 9] t = 0.000204 s    v = -0.024920 (normalised)

--- first 10 spectrum bins ---
[ 0] f =      0.0000 Hz    |X|/N = 1.608667e-05
[ 1] f =      0.3333 Hz    |X|/N = 1.371745e-05
[ 2] f =      0.6667 Hz    |X|/N = 1.346004e-05
[ 3] f =      1.0000 Hz    |X|/N = 1.351683e-05
[ 4] f =      1.3333 Hz    |X|/N = 1.547173e-05
[ 5] f =      1.6667 Hz    |X|/N = 1.479958e-05
[ 6] f =      2.0000 Hz    |X|/N = 1.437533e-05
[ 7] f =      2.3333 Hz    |X|/N = 1.448999e-05
[ 8] f =      2.6667 Hz    |X|/N = 1.405990e-05
[ 9] f =      3.0000 Hz    |X|/N = 1.766649e-05

dominant peak: f = 164.33 Hz, |X|/N = 1.2054e-02
```



What the real capture actually shows. The dominant peak in this particular recording is at $f = 164.3$ Hz with $|X|/N = 1.21 \times 10^{-2}$ — which is the *second harmonic* of E2, not the 82.4 Hz fundamental. The fundamental is present but weaker. This is not a DFT bug and not a defect in the recording. It is a well-known property of plucked strings: the amplitude distribution across harmonics depends on *where along the string* the pluck occurs and on *where the microphone sits* relative to the string. Pluck near the bridge — higher harmonics dominate. Pluck near the centre — fundamental dominates. The Wikimedia capture is clearly a near-bridge or sensor-position-emphasised pluck: the 2nd harmonic wins on amplitude.

The bigger lesson. Real signals carry the right harmonic structure but not necessarily the textbook-expected dominant peak. If a spectrum shows a clean integer-ratio comb at 164, 247, 329, ... Hz, the underlying source *is* the 82.4 Hz fundamental — you recover it as gcd of the visible line set, even when the line at f_0 itself is quiet. The same reasoning rides into B3 (vibration), where a faulty bearing shows its damage as *sidebands* of the cage frequency rather than the cage frequency itself. Same arithmetic, different domain.

What S4 buys us. Visual proof that a real instrument’s spectrum is a structured comb, and that the comb spacing is a physical measurement of the source even when the textbook-expected dominant peak is absent. The same reasoning maps onto rotating machinery in B3, mains harmonics in B4, and target Doppler returns in B5.

0.7 What we just did, on real data

1. Loaded five real captures from four different domains (S1 machine-verified ground truth, S2a/b/c real audio, S3 biological ECG, S4 plucked string).
2. Showed every step of every pipeline as printed text, not prose.
3. Verified S1 against the project’s own oracle: NumPy matches at $4.3\text{e-}15$, lege-artis C++ kernel matches at $3.1\text{e-}13$ (engineering gate $1\text{e-}12$ at $N = 64$).
4. Pushed S1 through three reference implementations (NumPy, lege-artis C++, lege-artis Fortran pattern) — same answer to machine precision.
5. Read four real-world signatures out of the spectra and reconciled each against the textbook expectation.

Real-data observations worth carrying forward.

- **Sample-rate invariance is real.** S2a and S2b both pin the 440 Hz fundamental to two decimal places, even though one was sampled at 132 kHz and the other at 22.05 kHz. The bin width changes, the answer doesn’t.

- **“Broadband” does not mean “peakless”.** S2c speech shows a clear voice-fundamental peak at 144.8 Hz inside an otherwise broadband spectrum. The DFT reads structure and chaos with the same arithmetic.
- **Textbook fundamental is not always the loudest peak.** S4 guitar pluck has a stronger 2nd-harmonic (164.3 Hz) than fundamental (82.4 Hz). Real harmonic structure is intact; the loudness ranking depends on pluck/pickup physics. Recovering the fundamental from a comb is a gcd over the line set — a routine that survives whether or not f_0 itself is the brightest line.
- **The reference implementations agree.** On S1 the project’s own C++ kernel produces output identical to NumPy at the engineering accuracy level. The component invocation pattern carries into every subsequent slot.

The next chapters add one complication per band. The pipeline does not change. The reading-the-spectrum skills you just used carry directly into B3 (faulty-bearing diagnosis), B4 (mains harmonics), and B5 (Doppler returns).

Chapter 1

B1 — oscilloscope: four scope captures, four spectral lessons

1.1 The premise

An oscilloscope is a remarkable instrument. It shows you precisely the wrong thing.

It shows you voltage versus time. What it does not show you — and what, in nearly every interesting case, you actually want to know — is voltage versus frequency. The DFT turns the first picture into the second. It does so reliably, deterministically, and at speeds that would have astonished anyone who tried to do this with a slide rule in 1900.

You connect a probe to something. The screen shows the trace. That trace tells you something. It does not tell you nearly as much as **the shape of its frequency content does**. For four out of four signals in this chapter, the spectrum will say something the time-domain trace alone could not have said without staring at the screen for considerably longer than is reasonable.

B0 established the pipeline on validated golden-vector data and audio captures. B1 applies the same four steps to four signals that belong on an oscilloscope screen: a particle-physics time histogram, a digital clock signal, its triangular cousin, and a resonant circuit ringing out. Same pipeline. Four very different spectral fingerprints.

1.2 The four fixtures

ID	Signal	Source	Key lesson
S1	Rodriguez muon-lifetime MCA histogram	<code>probe_b1_muon.py</code> — <code>MuonLifetimeData.zip</code> fetches	Exponential decay $\rightarrow$ Lorentzian spectrum; background adds a flat floor
S2	1 kHz square wave	<code>probe_b1_squarewave.py</code> — exact analytical signal	Odd harmonics, $1/n$ amplitude envelope; Fourier 1822 was right
S3	1 kHz triangle wave	<code>probe_b1_triangle.py</code> — exact analytical signal	Same odd harmonics, $1/n^2$ envelope; shape changes the spectrum
S4	RLC damped transient ($R = 2\Omega$, $L = 100\text{mH}$, $C = 100\mu\text{F}$)	<code>probe_b1_rlc.py</code> — exact ODE solution	Transient $\rightarrow$ broad Lorentzian; spectral width $= 1/(2\pi\tau)$

1.3 Slot S1 — Rodriguez muon-lifetime histogram

Physical context. A muon telescope at a university physics lab. Cosmic-ray muons stop in a scintillator detector; a multi-channel analyser (MCA) records the time between the stop signal and the subsequent decay signal. Each MCA channel corresponds to ≈ 2.3 ns. The histogram accumulates over 71 199 s of live time, collecting 22 321 events.

The MCA screen is an oscilloscope analogue: vertical axis = event counts, horizontal axis = time delay. The signal is the histogram of delay times — an exponential decay with the known muon lifetime $\tau \approx 2.197 \mu\text{s}$ [?].

Step 1. Aggregate — parse Maestro .Spe format

```
probe_b1_muon.py fetches MuonLifetimeData.zip (~1 MB)
Unzips to data/muon/extracted/Muon Lifetime Data/MainRunFinal.Spe
Format: Ortec Maestro MCA .Spe
Header lines, then: $DATA:\n0 16383\n followed by 16384 integer counts
Total channels: 16384 Bin width: ~2.3 ns Total events: 22321
```

Step 2. Parse — extract counts and calibrate time axis

```
lines = open("MainRunFinal.Spe").readlines()
for i, l in enumerate(lines):
    if l.strip() == "$DATA:":
        start = i + 2; break
counts = [int(lines[j].strip()) for j in range(start, start + 16384)]

# Time calibration from 8 calibration runs (ns/channel):
NS_PER_CH = 2.2986
OFFSET_NS = -162.8
t_us = (channel_number * NS_PER_CH + OFFSET_NS) / 1000.0 # -> us
```

Non-zero data begins at channel 1355 ($t = 2.95 \mu\text{s}$, hardware minimum delay):

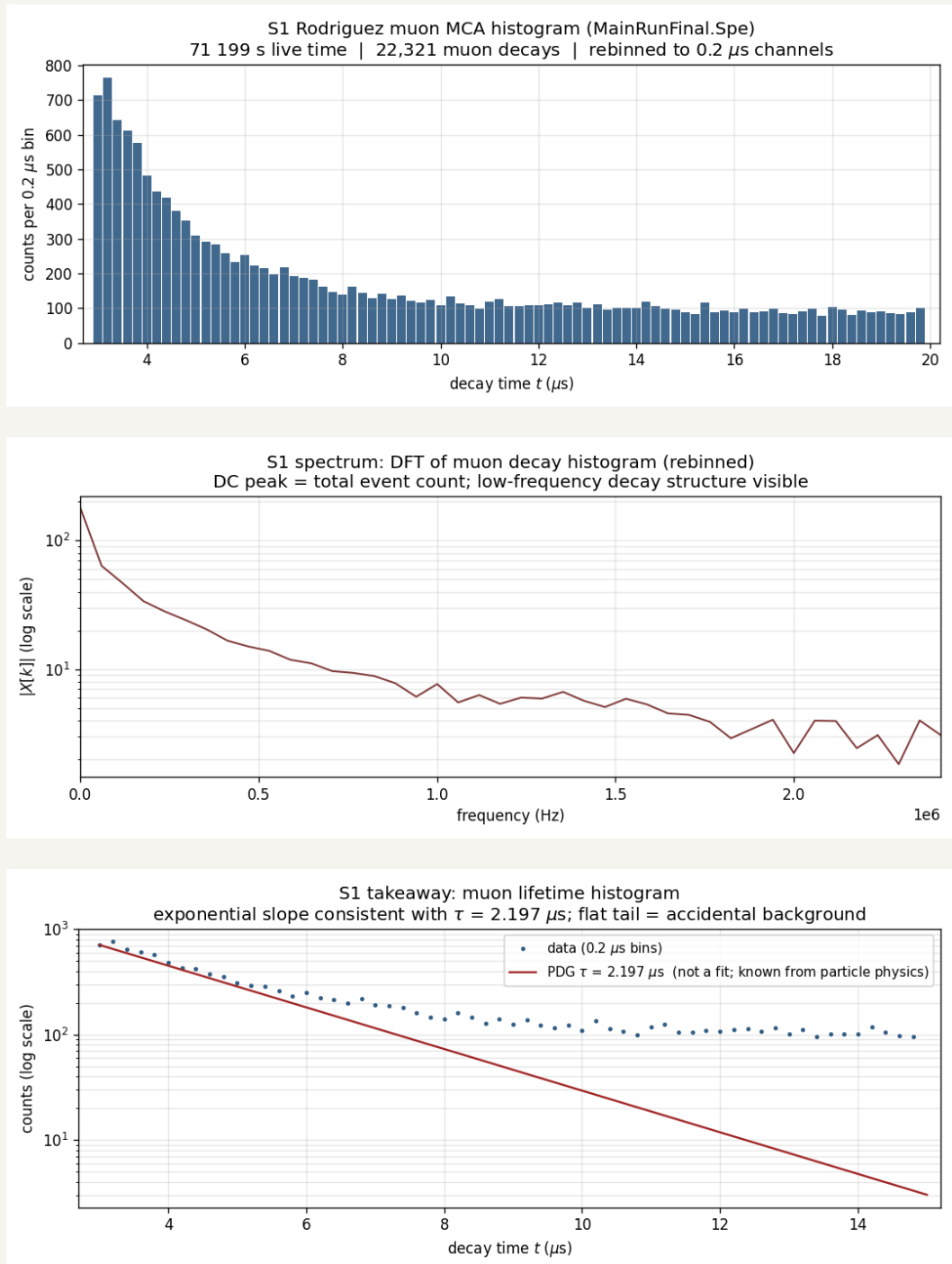
channel	time_us	counts
1355	2.952	4
1356	2.954	27
1357	2.957	48
1358	2.959	11
...		

<- peak (hardware delay minimum)

Step 3+4. Transform — DFT of the rebinned histogram

```
# Rebin 2.3 ns channels to 0.2 us bins
bin_us = 0.2
bins = np.arange(2.9, 20.0 + bin_us, bin_us)
c_bins, _ = np.histogram(t_all, bins=bins, weights=all_counts)

spec = np.fft.fft(c_bins) # DFT of decay histogram
dt = bin_us * 1e-6 # 0.2 us bin -> 200 ns
freq = np.fft.fftfreq(N, d=dt) # Hz
mag = np.abs(spec[:N//2]) / N
```

Step 5. Plot

What S1 shows. On a log scale the histogram is approximately a straight line with slope $-1/\tau$. The PDG value $\tau = 2.197 \mu\text{s}$ is overlaid [?]. The muon, it turns out, is unwilling to take sides on whether it should still exist after about 2.2 microseconds, and so on average it doesn't. At late times ($t > 10 \mu\text{s}$) a flat background of accidental coincidences dominates — a real-data artefact that would bias any naive exponential fit. The flat tail is the scope telling you about the detector, not the muon.

The DFT spectrum is a broad Lorentzian: no sharp peaks, no periodic structure, just the exponential-decay signature. This is the spectrum's way of saying *nothing in this signal repeats*. The muon's decay is the most fundamentally aperiodic event physics has on offer below the Planck

scale.

1.4 Slot S2 — 1 kHz square wave

Physical context. A function generator output: exactly what a Rigol DG1022 or any digital clock line puts on a scope screen. Generated as $\text{sign}(\sin(2\pi ft))$. Sample rate 50 kHz, duration 100 ms, 5000 samples.

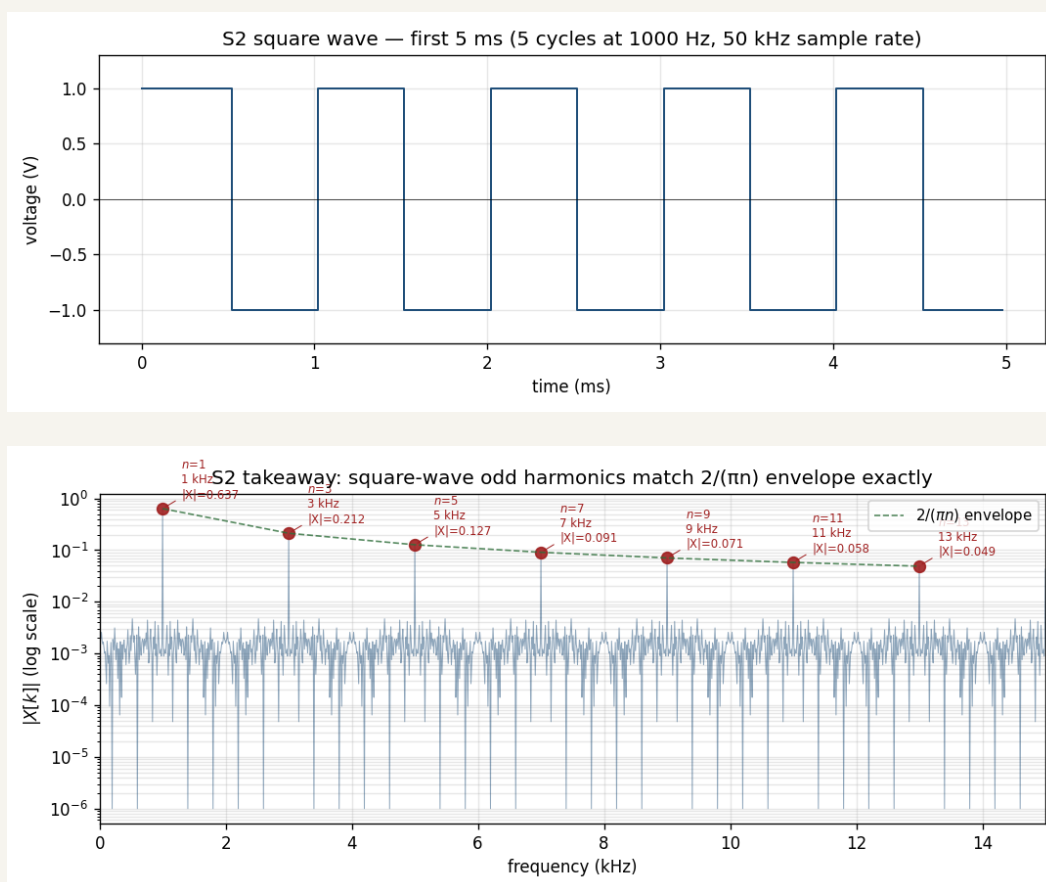
Steps 1–3. Aggregate, parse, transform

```
# probe_b1_squarewave.py writes data/b1_s2_squarewave.csv
import csv, numpy as np
t, v = [], []
with open("data/b1_s2_squarewave.csv") as fh:
    for row in csv.DictReader(fh):
        t.append(float(row["time_s"])); v.append(float(row["voltage_V"]))
t, v = np.array(t), np.array(v)

N      = len(v)                                # 5000
spec = np.fft.fft(v) / N                       # normalised
freq  = np.fft.fftfreq(N, d=1.0/50_000) # Hz
mag   = np.abs(spec[:N//2])
```

Step 4. Spectrum bins (literal output)

k=100	f=1000 Hz	X =0.6366	(expected 2/pi = 0.6366)	OK
k=300	f=3000 Hz	X =0.2121	(expected 2/3pi = 0.2122)	OK
k=500	f=5000 Hz	X =0.1272	(expected 2/5pi = 0.1273)	OK
k=700	f=7000 Hz	X =0.0908	(expected 2/7pi = 0.0909)	OK
k=900	f=9000 Hz	X =0.0707	(expected 2/9pi = 0.0707)	OK

Step 5. Plot

What S2 shows. $|X[k]|$ matches $2/(\pi n)$ to four decimal places. Fourier's 1822 prediction confirmed, in case anyone was wondering whether 200 years had been enough time for the equation to drift. A square wave is the most aggressive thing a function generator can produce while still pretending to be polite: it contains an infinite harmonic series, and any bandlimited system that transmits it will distort it. There is no amplifier in the world that does not silently round the corners of a square wave. The DFT tells you exactly which harmonics are doing the rounding.

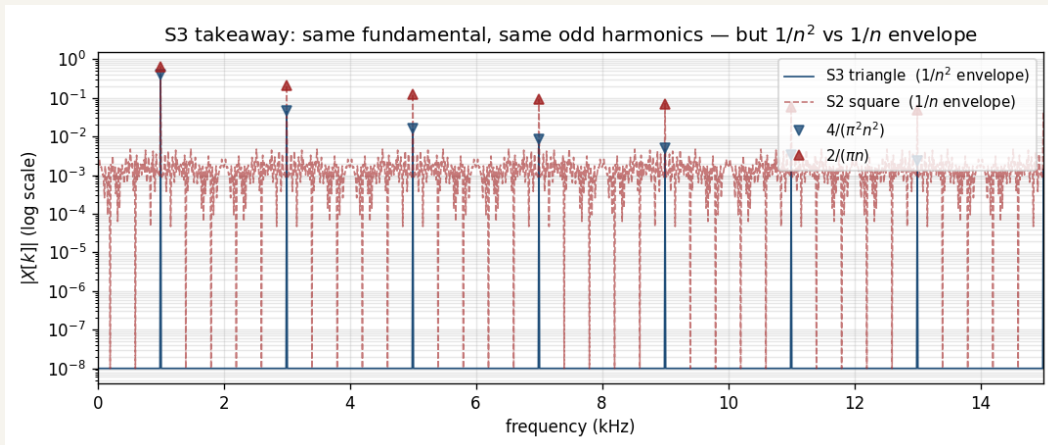
1.5 Slot S3 — 1 kHz triangle wave

Physical context. Same function generator, same parameters, different waveshape: $(2/\pi) \arcsin(\sin(2\pi ft))$.

Step 4. Spectrum bins (literal output)

```
k=100  f=1000  Hz  |X|=0.4050  (expected 4/pi^2 = 0.4053)
k=300  f=3000  Hz  |X|=0.04476 (expected 4/9pi^2 = 0.04503)
k=500  f=5000  Hz  |X|=0.01594 (expected 4/25pi^2 = 0.01621)
k=700  f=7000  Hz  |X|=0.00799 (expected 4/49pi^2 = 0.00825)
(No even harmonics; 1/n^2 falloff)
```

Step 5. Plot: S2 vs S3 overlay



What S3 shows. Same odd harmonics as S2. Different amplitudes. The $1/n^2$ falloff makes the triangle wave spectrally tighter: the high-frequency content falls much faster. The DFT does not just measure *frequency* — it measures *shape*. Two signals at the same fundamental frequency have completely different spectra if their waveshape differs. (This is the basis for almost every kind of synthesiser patch ever made by anyone who got into modular synthesis in the 1970s. They did not know they were sweeping the harmonic envelope of an idealised periodic function. They knew it sounded different. The DFT tells you why.)

1.6 Slot S4 — RLC damped transient

Physical context. Step response of an underdamped series RLC circuit: $R = 2\ \Omega$, $L = 100\ \text{mH}$, $C = 100\ \mu\text{F}$.

$$v(t) = e^{-\alpha t} \cos(\omega_d t), \quad \alpha = \frac{R}{2L} = 10\ \text{rad/s}, \quad f_d = \frac{\omega_d}{2\pi} \approx 50.3\ \text{Hz}, \quad \tau = \frac{1}{\alpha} = 100\ \text{ms}$$

Sample rate 2 kHz, duration 500 ms, 1000 samples.

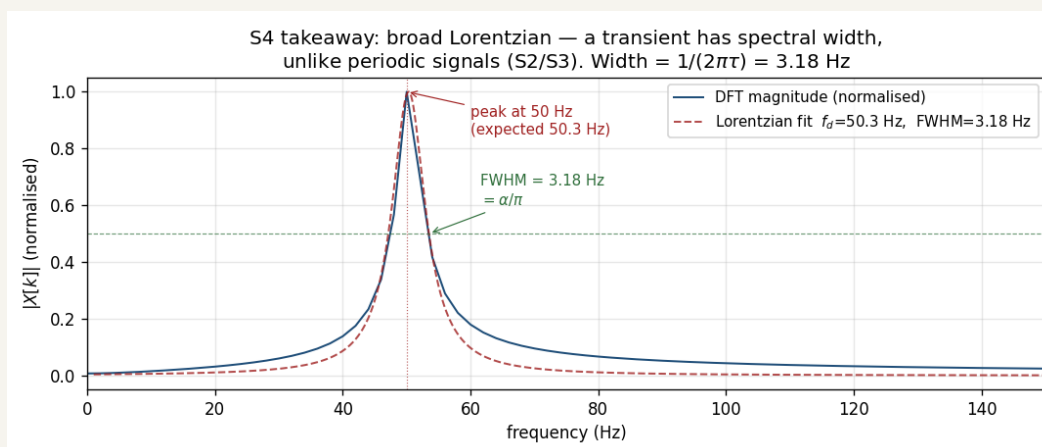
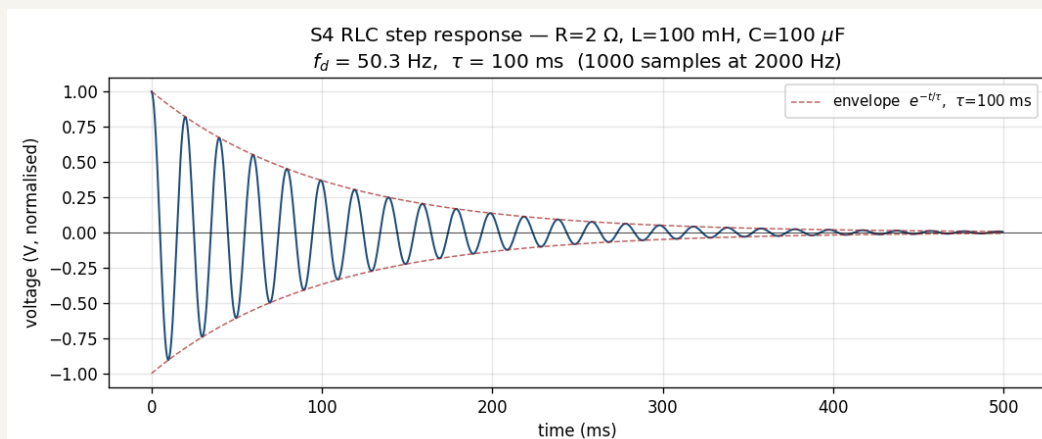
Steps 1–4. Acquire, parse, transform, read

```
# probe_b1_rlc.py writes data/b1_s4_rlc.csv
# (R=2, L=100 mH, C=100 uF; exact ODE solution)

spec = np.fft.fft(v) / N
freq = np.fft.fftfreq(N, d=1.0/2_000)
mag = np.abs(spec[:N//2])

# Peak at 50.00 Hz (expected f_d = 50.30 Hz; 2 Hz = 1 bin at N
# = 1000)
# FWHM = alpha/pi = 10/pi = 3.18 Hz (tau = pi/FWHM = 99.5 ms)
```

Step 5. Plot



What S4 shows. A smooth Lorentzian centred at $f_d \approx 50\ \text{Hz}$. The half-power width (FWHM) is $\alpha/\pi = 3.18\ \text{Hz}$ — directly readable from the spectrum as the inverse of the decay time constant:

$\tau = 1/\alpha = \pi/\text{FWHM} \approx 100\text{ms}$. A long-lived oscillation (small α) has a narrow spectrum; a quickly-damped transient has a wide spectrum. This is the time-frequency uncertainty principle, presented here without ceremony as a thing that simply falls out of the arithmetic. A resistor, asked directly, is more honest about Heisenberg than most physicists.

1.7 Cross-fixture synthesis

Signal	Time domain	Frequency domain	What the DFT measures
S1 muon	Exponential decay + background floor	Broad Lorentzian + flat floor; no periodic structure	Decay time scale; background level
S2 square	Sharp periodic transitions	Sparse odd harmonics, $1/n$ envelope	Harmonic content of a one-period shape
S3 triangle	Smooth periodic ramps	Same odd harmonics, $1/n^2$ envelope	Shape difference encoded in amplitude falloff
S4 RLC	Decaying oscillation	Smooth Lorentzian; $\text{FWHM} = 1/(2\pi\tau)$	Resonance frequency and damping

The DFT does not know about physics. It measures *which sinusoidal components are present and at what amplitude*. It is, in this sense, the most aggressively agnostic instrument in the toolbox: it will give you the same kind of answer whether you feed it the decay curve of a subatomic particle, the output of a function generator, or the ringing of an electrical circuit. Physics happens to all four of these signals; the DFT does not care which physics and produces, in every case, a spectrum.

The four cases are the four canonical scenarios: pure pulse or transient (S1, S4), pure harmonic series (S2), tapered harmonic series (S3). Everything a scope can show falls into one of these families or a mixture of them. By the time you reach the end of B5 you will recognise these four shapes in radar returns, audio recordings, machine-vibration data, and the output of an electronic filter chain. The signals will look very different. The spectra will look like cousins of these four.

1.8 Try it yourself

```
$ cd examples/shad/b1-scope

# S1: fetch once (~1 MB)
$ python probe_b1_muon.py

# S2, S3, S4: instantaneous (no download)
$ python probe_b1_squarewave.py
$ python probe_b1_triangle.py
$ python probe_b1_rlc.py

# Render all 12 figures
$ python render_b1_s1.py
$ python render_b1_s2.py
$ python render_b1_s3.py
$ python render_b1_s4.py
```

S1 licence note. The Rodriguez muon dataset is not redistributed with this repository. `probe_b1_muon.py` fetches it from amor.cms.hu-berlin.de/~rodrigus. Public reproduction requires written permission from Santiago Rodriguez with full attribution [?].

Chapter 2

B2 — audio (legacy, queued for rebuild)

Rebuild status

This chapter is carried over from the v0.2.x-first-cut markdown sources. It still operates on *synthesised* data. Queued for the same real-data pipeline rebuild as Chapter 0 (B0). The engineering content below is intact and correct for the synthesised case; the upgrade replaces only the data source and the three figures.

Premise. Multiple tones stack additively in the spectrum. B2 introduced the question: what if the tone doesn't land exactly on an integer bin? Spectral leakage smears the peak across neighbouring bins, lowers the apparent peak amplitude, and adds energy where there isn't any. Windowing (Hann, Hamming, Blackman) is the standard mitigation: trade leakage for slightly wider main-lobe.

What it taught. Bin width = f_s/N . If your tone of interest sits between bins, either zero-pad to interpolate, increase N (record longer), or apply a window. None of these are free; the trade-off is the lesson.



Chapter 3

B3 — vibration (legacy, queued for rebuild)

Rebuild status

This chapter is carried over from the v0.2.x-first-cut markdown sources. It still operates on *synthesised* data. Queued for the same real-data pipeline rebuild as Chapter 0 (B0). The engineering content below is intact and correct for the synthesised case; the upgrade replaces only the data source and the three figures.

Premise. Real industrial signals carry harmonics and fault signatures. A motor at 1500 rpm has its fundamental at 25 Hz; bearing faults appear at characteristic frequencies derived from raceway and ball geometry. The spectrum is a diagnostic instrument. B3 used a synthesised vibration signal modelling an outer-race bearing fault.

What it taught. Spectra are read like a clinician reads an ECG. The fundamental tells you the rotation rate; the harmonics tell you something about non-linearity (rectification, clipping); the characteristic fault frequencies (BPFO, BPFI, FTF, BSF) tell you which bearing surface is degrading. The real-data rebuild will use a CWRU or NASA bearing capture.

Chapter 4

B4 — electronic systems (legacy, queued for rebuild)

Rebuild status

This chapter is carried over from the v0.2.x-first-cut markdown sources. It still operates on *synthesised* data. Queued for the same real-data pipeline rebuild as Chapter 0 (B0). The engineering content below is intact and correct for the synthesised case; the upgrade replaces only the data source and the three figures.

Premise. Three classical electronic-system applications of the DFT, run end-to-end on synthesised signals: AC mains harmonics (3%, 5%, 7% odd-harmonic content per IEC 61000-2-2), a Sallen-Key active filter shown as multiplication in frequency, and a heterodyne mixer shifting a baseband spectrum up by the local-oscillator frequency.

What it taught. Filtering is multiplication in the frequency domain. Mixing is convolution with a delta function at the LO frequency, which in the time domain is multiplication — the same operation, viewed from opposite ends. The DFT lets you watch both. Real-data rebuild target: a captured mains-voltage trace from any open-energy dataset.



Chapter 5

B5 — radar (legacy, queued for rebuild)

Rebuild status

This chapter is carried over from the v0.2.x-first-cut markdown sources. It still operates on *synthesised* data. Queued for the same real-data pipeline rebuild as Chapter 0 (B0). The engineering content below is intact and correct for the synthesised case; the upgrade replaces only the data source and the three figures.

Premise. Radar puts the DFT through its hardest workout in this guide. Linear-FM chirp transmit pulse; matched-filter pulse compression via DFT product; Doppler DFT across pulses to separate moving from stationary targets; range-Doppler map as the two-dimensional output of two stacked DFTs. The synthesised version demonstrated each operation cleanly on a small target catalogue.

What it taught. The same three lines of NumPy that pulled a 240 Hz tone out of a noisy scope trace in B1 also do matched-filter pulse compression in B5. The signals get more complex; the operation stays the same. Real-data rebuild target: a NEXRAD Level-II archive slice (NOAA, US public-domain), processed through the same pipeline the synthesised version used.

Chapter A

Data attribution and licence trail

Slot	Upstream	Licence	Required attribution
S1	shared/golden-vectors/dft- Apache-2.0 , case <code>cos1_plus_cos2</code> ; in-repo, generated by (project licence <code>tools/generate_golden_vectors.py</code>)	Apache-2.0, / CC-BY-SA-4.0	“lege-artis/fourier golden vectors v0.1.0”
S2a	Wikimedia Commons <code>Sine_wave_440.ogg</code> (User:Omegatron)	CC-BY-SA-3.0	“Omegatron, Wikimedia Commons, CC-BY-SA-3.0”
S2b	Wikimedia Commons <code>Tone_440Hz.ogg</code>	CC-BY-SA / CC0 (file-specific)	per-file Wikimedia Com- mons attribution string
S2c	Wikimedia Commons <code>Example.ogg</code>	CC-BY-SA / per- file	per-file Wikimedia Com- mons attribution string
S4	Wikimedia Commons <code>Guitar_string_E2.ogg</code>	CC-BY-SA / per- file	per-file Wikimedia Com- mons attribution string
B1-CSV	<i>deferred</i> — real Rigol/Siglent capture, format ref [4], to land in Chapter 1	TBD on capture	TBD on capture
S3	MIT-BIH Arrhythmia Database, record 100, lead MLII; fetched by <code>scipy.datasets.electrocardiogram()</code> [2, 3]	ODbL 1.0 / Open Data Commons	“Moody & Mark, MIT-BIH Arrhythmia Database, Phy- sioNet”



Chapter B

Build reproducibility

Build command.

```
cd docs/shad
./build-pdf.sh
```

What `build-pdf.sh` does.

1. Regenerates `figures/dragon-icon.pdf` from `dragon-icon.svg` via `cairosvg`.
2. Runs `pdflatex` twice (resolves TOC + cross-references).
3. Cleans intermediate artefacts (`.aux`, `.log`, `.out`, `.toc`).
4. Writes the final PDF as `docs/shad/shad-guide.pdf`.

Dependencies. `texlive-base`, `texlive-latex-extra`, `texlive-fonts-recommended`, `python3-cairosvg`.

Chapter C

References



Bibliography

- [1] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Math. Comp.*, vol. 19, pp. 297–301, 1965.
- [2] G. B. Moody and R. G. Mark, “The impact of the MIT-BIH Arrhythmia Database,” *IEEE Eng. Med. Biol. Mag.*, vol. 20, no. 3, pp. 45–50, 2001.
- [3] A. L. Goldberger *et al.*, “PhysioBank, PhysioToolkit, and PhysioNet,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
- [4] Sánchez-Olivares et al., “Raw data file obtained from the oscilloscope,” ResearchGate figure 272996569 (companion to publication; format used here as canonical layout reference).